

AddisCoder: Week 2, Day 3

Getting the Hang of Recursion

Taught by Huy Nguyen (huynguyen@gmail.com).

To find the value to the following expression 2^n for non-negative integers n , we can use an old friend `for` loop.

```
# Iterative solution (means using a loop)
# base ^ 4 = base * base * base * base
def exp(base, exponent):
    total = 1
    for i in range(exponent):
        total = total * base
    return total

print(exp(2, 10))

1024
```

But we can also use **recursion** to help find the value for the same expression. When solving a problem in programming, often times, multiple solutions to the same problem exist.

BASE CASE

The base case returns the answer to the smallest version of the problem.

For our example, the smallest version of the problem would be finding the answer to 2^0 .

```
if exponent == 0:
    return 1
```

RECURSION STEP

The recursion step solves smaller version of the problem and combines answers at each step of the smaller problem to return the final answer.

For our example, knowing the answer to 2^4 would help find the answer to 2^5 , knowing the answer to 2^3 would help find the answer to 2^4 , ...

```
small_answer = exponentiate_recur(base, exponent - 1)
return 2 * small_answer
```

Tracing steps

`exponentiate_recur(2, 5)` => recursion step =>

`2 * exponentiate_recur(2,4)` => recursion step =>

`2 * 2 * exponentiate_recur(2,3)` => recursion step =>

`2 * 2 * 2 * exponentiate_recur(2,2)` => recursion step =>

`2 * 2 * 2 * 2 * exponentiate_recur(2,1)` => recursion step =>

`2 * 2 * 2 * 2 * 2 * exponentiate_recur(2,0)` => base case =>

`2 * 2 * 2 * 2 * 2 * 1` => recursion step (to combine answers) =>

32

```
# Let's implement it recursively!
# base ^ 4 = base * (base * base * base) = base * base ^ 3
def exp_recursive(base, exponent):
    print("exp_recursive(" + str(base) + "," + str(exponent) + ")")
    if exponent == 0:
        # base case
        return 1
    else:
        # recursive case
        partial_sol = exp_recursive(base, exponent - 1)
        return base * partial_sol

# Equivalent code but shorter solution:
# def exp_recursive(base, exponent):
#     if exponent == 0:
#         return 1
#     return base * exp_recursive(base, exponent - 1)

print(exp_recursive(2, 10))

exp_recursive(2,10)
exp_recursive(2,9)
exp_recursive(2,8)
exp_recursive(2,7)
exp_recursive(2,6)
exp_recursive(2,5)
exp_recursive(2,4)
exp_recursive(2,3)
exp_recursive(2,2)
exp_recursive(2,1)
exp_recursive(2,0)
0.0009765625
```

```

# Class Question:
# how to check if a number is a power of 3:

# 9 = 3 * 3
# 27 = 3 * 3 * 3
# 81 = 3 * 3 * 3 * 3
# Notice that pattern: Any number n that is a power of 3, has (n/3)
also be a power of 3.  <---- Recursive case!
# The simplest power of three we know is 3**0 = 1, so 1 is the
simplest power of 3 that we know.
def is_power_of_3(num):
    print(num)
    if num == 1:
        return True
    elif num < 1:
        return False
    else:
        result = num / 3
        return is_power_of_3(result)

# Same logic as above but shorter.
# def is_power_of_3(num):
#     if num <= 1:
#         return num == 1
#     return is_power_of_3(num / 3)

is_power_of_3(81)

81
27.0
9.0
3.0
1.0

True

```

Branching Recursion

Sometimes, recursion can *branch*.

The following sequence of numbers is called the **Fibonacci numbers**:

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
```

Can you see the pattern in this sequence?

The **Fibonacci numbers** are defined as follows:

```
F(0) = 0
F(1) = 1
F(n) = F(n - 1) + F(n - 2), for n >= 2
```

We can expand this out a bit to see the first few **Fibonacci numbers** in the sequence:

```
F(0) = 0, by definition, (0-th Fibonacci number)
F(1) = 1, by definition, (1-st Fibonacci number)
F(2) = F(1) + F(0) = 1 + 0 = 1, (2-nd Fibonacci number)
F(3) = F(2) + F(1) = 1 + 1 = 2, (3-rd Fibonacci number)
F(4) = F(3) + F(2) = 2 + 1 = 3, (4-th Fibonacci number)
...
```

This sequence appears in algorithms, math, and in nature in various places. Today, we can use the definition of Fibonacci numbers to program a function that returns the n -th Fibonacci number!

Let us define a function `fib(n)` that takes a non-negative integer n as an input and outputs the n^{th} Fibonacci number.

```
def fib(n):
    # Fib Index      : 0  1  2  3  4  5  6  7  8
    # Fib sequence:  0  1  1  2  3  5  8 13 21
    # iterative      ^  *
    # n >= 1
    def fib_iter(n):
        fib_prev = 0
        fib_curr = 1

        for i in range(n):
            new_fib_curr = fib_curr + fib_prev
            fib_prev = fib_curr
            fib_curr = new_fib_curr

        return fib_prev

    for i in range(0,20):
        print(fib_iter(i), end=" ")

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

7 lines of code!

Try, on your own, writing what a recursive method for finding the n -th Fibonacci number might look like

```
# implementation of fib using recursion:
```

```
# Look how pretty and easy to understand
def fib_recursive(n):
    # If n == 0, returns 0. If n == 1, returns 1.
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n - 2)
```

3 lines of code!!

While recursion solutions can be more difficult to think about at first, they can be simpler and more elegant solutions to a problem.

```
# Let's try running it with a few different values of n:
```

```
print(fib_recursive(0))
```

```
0
```

```
print(fib_recursive(5))
```

```
5
```

```
# Hmm, that's interesting. It takes a while to run.
```

```
print(fib_recursive(35))
```

```
9227465
```

```
# The iterative implementation seems much faster?
```

```
print(fib_iter(10000))
```

```
3364476487643178326662161200510754331030214846068006390656476997468008
1442166662368155595513633734025582065332680836159373734790483865268263
0408924630564318873545443695598274916066020998841839338646527313000888
3026923567361313511757929743785441375213052050434770160226475831890652
7890855154366159582987279682987510631200575428783453215515103870818298
9697916131278562650331954871402142875326981879620469360978799003509623
0229102636813149319527563022783762844154036058440257211433496118002309
1208287046088923962328835461505776583271252546093591128203925285393434
6209042452489294039017062338889910858410651831733604374707379085526317
6432573399371287193758774689747992630583706574283016163740896917842637
8624212835258112820516370298089332099905707920064367426202389783111470
0540749984592503606335609338838319233867830561364353518921332797329081
3373264265263398976392272340788292817795358057099369104917547080893184
1056146322338217465637321248226383092103297701648054726243842374862411
4530938122065649140327510866433945175121615265453613331113140424368548
0510676584349352383695965342807176877532834823434555736671973139274627
3629108210679280784718035329131176778924659089938635459327894523777674
4061922403376386740040213303432974969020283281459334188268176838930720
0363479562311710310129195316979460763273758925353077255237594378843450
4067715555779056450443016640119462580972216729758615026968443146952034
```

```
6149322911059706762432685159928347098912847067408620085871350162603120
7190317208609408129832158107728207635318662461127824553720853236530577
5956430072517744315051539600905168603220349163222640885248852433158051
5348496224348482993809050704834824493274537326245677558790891871908036
6205800959474315005240253270974699531877072437682590741993963226598414
7498193609285223945039707165443156421328157688908058783183404917434556
2705202235648464951961124602683139709750693826487066132645076650746115
1267752274862159864253071129844118262266105716351506926002986170494542
5047491378115154139941550671256271197133252763631939606902895650288268
608362241082050562430701794976171121233066073310059947366875
```

Extra Practice Problems

```
# Question 1: Skip Add
# Write a function skip_add that takes a single argument n and
# computes the sum of every other integers between 0 and n starting from
# n. Assume n is non-negative.
def skip_add(n):
    """ Takes a number x and returns x + x-2 + x-4 + x-6 + ... + 0.

    >>> skip_add(5) # 5 + 3 + 1 + 0
    9
    >>> skip_add(10) # 10 + 8 + 6 + 4 + 2 + 0
    30
    """
    if n <= 1:
        return n
    else:
        return n + skip_add(n-2)

print(skip_add(5))
print(skip_add(10))

9
30

# Question 2: Common Misconception
# Fix the error with this recursive function.
# We will debug this using the recursion tree.
def skip_mul(n):
    """Return the product of n * (n - 2) * (n - 4) * ...

    >>> skip_mul(5) # 5 * 3 * 1
    15
    >>> skip_mul(8) # 8 * 6 * 4 * 2 * 0
    0
    """
    if n == 0:
        return 0
    elif n == 1:
```

```

        return 1
    else:
        return n * skip_mul(n - 2)

# Question 3: Common Misconception
# Fix the error with the following recursive function:
def print_up_to(n):
    """Print every natural number up to n, inclusive.

    >>> print_up_to(5)
    1
    2
    3
    4
    5
    """
    i = 1
    if i > n:
        return
    else:
        print(i)
        i += 1
        print_up_to(n)

# Question 4: Hailstone
# For the hailstone function, you pick a positive integer n as the
# start. If n is even, divide it by 2. If n
# is odd, multiply it by 3 and add 1. Repeat this process until n is
# 1. Write a recursive version of hailstone that prints out
# the values of the sequence and returns the number of steps.
def hailstone(n):
    """Print out the hailstone sequence starting at n, and return the
    number of elements in the sequence.

    >>> a = hailstone(10)
    10
    5
    16
    8
    4
    2
    1
    >>> a
    7
    """
    print(n)
    if n % 2 == 0:
        return 1 + hailstone(n//2)
    elif n != 1:
        return 1 + hailstone(n*3+1)

```

```

        else:
            return 1

a=hailstone(10)
print(a)

10
5
16
8
4
2
1
7

# Question 5:
# Given a value n, produce a list of all different ways to write n as
the sum of positive non-increasing terms
# Example: n=4
# The function should produce the list
# [[1, 1, 1, 1],
# [2, 1, 1],
# [2, 2],
# [3, 1],
# [4]]

def writing_as_sum_attempt1(n):
    #????
    pass

# We can try to choose the first term in the sum, and then write the
rest as a sum. However, we now have the
# restriction that the rest of the terms have to be smaller than the
first term.
# This means we need to keep an extra state to keep track of the
maximum value that can be used.

def writing_as_sum(n, v):
    """List all different ways to write n as a sum of positive non-
increasing terms
and furthermore, all the terms are at most v
"""
    if n==0:
        return [[]]
    answer = []
    for u in range(1, min(v + 1, n + 1)):
        tmp = writing_as_sum(n - u, u)
        for lst in tmp:
            answer.append([u] + lst)
    return answer

```

```
lst = writing_as_sum(4, 4)
print(lst)
```

```
[[1, 1, 1, 1], [2, 1, 1], [2, 2], [3, 1], [4]]
```